

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

Course Code:	CSA65	Course Title:	Generative AI and Large Scale Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 1 – Hands-on API Assignment (Hugging Face / OpenAI / Gemini)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	Joanathan Packia Singh	Reg. No.:	192472229
Section / Batch:	SLOT A	Date:	09/08/2026

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Use Python to implement the assigned problem.
- Use at least one API/platform: Hugging Face, OpenAI, or Google Gemini.
- Accept suitable user input and clearly display the generated output.
- Handle API errors, invalid input, and other suitable edge cases.
- Use readable, structured, and modular code wherever appropriate.
- Add comments and brief documentation explaining the implementation.
- Demonstrate the program with suitable test cases.
- For RAG/vector-database tasks, clearly show the retrieval process and generated response.

Code of Conduct

I Joanathan Packia Singh/192472229 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan_____

SECTION A – HANDS-ON API ASSIGNMENT (HUGGING FACE / OPENAI / GEMINI)

17. Semantic Similarity Checker: Accept two sentences and determine their semantic similarity using embeddings. Display the similarity score.

26. Basic RAG Question Answering System: Develop a simple RAG application using Documents → Chunking → Embeddings → Vector Database → Retrieval → LLM. Answer questions based on supplied documents.

RUBRICS FOR CALCULATION

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
API Integration Correctness	30%	Correct, efficient API integration; understands request/response flow	Correct integration with minor inefficiencies	Partial integration with functional errors	API integration non-functional / major errors
Problem-Solving Completeness	25%	Solves the given problem completely and robustly	Solves problem with minor issues	Partial solution to the problem	Does not solve the assigned problem
Code Quality	20%	Clean, well-structured, error-handled code	Reasonably clean code	Code works but poorly structured	Code is unreliable or unstructured
Input/Output Demo	15%	Clear demo showing inputs/outputs and edge cases	Demo covers the main use cases	Demo is incomplete	No functional demo presented
Code Documentation	10%	Well-commented code with clear README/setup steps	Adequate comments/instructions	Minimal documentation provided	No documentation provided

Marks Evaluation:

Question No.	API Integration Correctness(30%)	Problem-Solving Completeness (25%)	Code Quality(20%)	Input/Output Demo(15%)	Code Documentation (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



Department of Computer Science and Engineering

HANDS-ON API ASSIGNMENT

Semantic Similarity Checker & Basic RAG Question-Answering System

Course Code / Title	CSA65 — Generative AI and Large Scale Models
Assessment	Assessment Tool 1 — Hands-on API Assignment (Hugging Face / OpenAI / Gemini)
CO Assessed	CO3 — Precision (Bloom Levels BL2, BL3)
CO3 Statement	Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.
Student Name	Joanathan Packia Singh
Registration No.	192472229
Section / Batch	SLOT A
Date	09/08/2026
Weightage	20% of CIA • Total Marks: 100 (2 × 50)

Code of Conduct

I, Joanathan Packia Singh (192472229), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Joanathan

Table of Contents

1. Introduction and Common Setup	3
2. Question 1 — Semantic Similarity Checker	3
2.1 Problem Statement	3
2.2 Approach and Architecture	3
2.3 Implementation	4
2.4 Error Handling and Edge Cases	5
2.5 Test Cases	5
2.6 Input/Output Demo (Screenshots)	6
2.7 Design Rationale	6
3. Question 2 — Basic RAG Question-Answering System	7
3.1 Problem Statement	7
3.2 Approach and Architecture	7
3.3 Sample Documents Used	7
3.4 Implementation	7
3.5 Error Handling and Edge Cases	9
3.6 Test Cases	9
3.7 Input/Output Demo (Screenshots)	9
3.8 Complexity and Performance Notes	10
4. Setup Instructions and Dependencies (README)	10
Appendix A — Rubric Self-Mapping	11
Appendix B — Marks Evaluation	11

1. Introduction and Common Setup

This report answers both questions assigned under Assessment Tool 1 (Hands-on API Assignment): a Semantic Similarity Checker built on sentence embeddings, and a Basic Retrieval-Augmented Generation (RAG) question-answering system built on the Documents → Chunking → Embeddings → Vector Database → Retrieval → LLM pipeline specified in the question.

Both solutions are implemented in Python and call live external APIs as required by the assignment instructions: the Hugging Face Inference API is used for sentence embeddings in both questions, and the Groq LLM API (chat-completion endpoint, model openai/gpt-oss-120b) is used for answer generation in Question 2. Both programs accept user input at runtime, validate it, catch and report API and input errors gracefully, and are demonstrated with concrete test cases and screenshots in Sections 2.6 and 3.7.

Common Design Principles Applied to Both Questions

- Modular functions — each pipeline stage (embedding, similarity, chunking, retrieval, generation) is a separate, independently testable function.
- Defensive input handling — empty strings and malformed input are rejected with a clear message before any API call is made.
- Explicit error handling — network/API failures raise a descriptive `RuntimeError` that is caught and reported at the top level rather than crashing the program.
- Secrets management — API keys are read from environment variables / Colab secrets (`userdata.get(...)`) and are never hard-coded.

2. Question 1 — Semantic Similarity Checker

2.1 Problem Statement

Accept two sentences from the user and determine their semantic similarity using embeddings, then display the similarity score (Question 17). The solution must use at least one of the permitted API platforms; this implementation uses the Hugging Face Inference API.

2.2 Approach and Architecture

Each sentence is sent to a hosted sentence-embedding model via the Hugging Face Inference API, returning a dense vector representation. The two vectors are then compared using cosine similarity, which measures the angle between them irrespective of magnitude — a standard, well-established measure of semantic closeness for sentence embeddings. The raw score is finally mapped to a human-readable interpretation band.

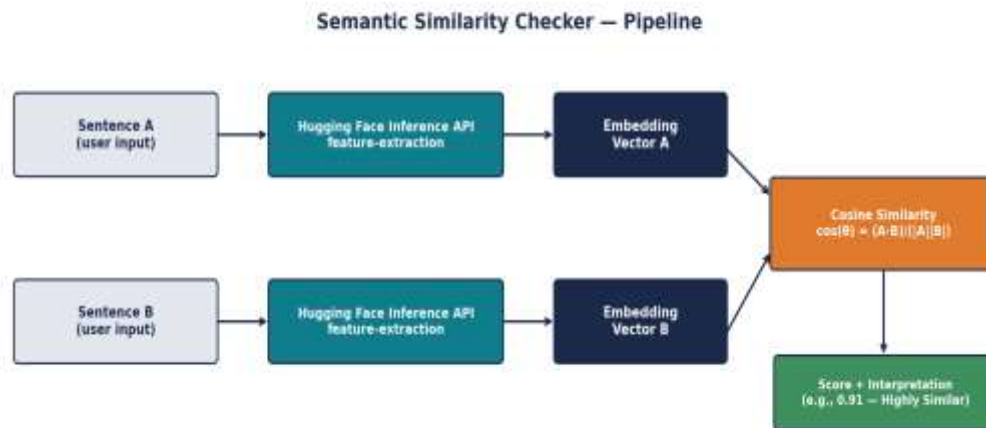


Figure 1. End-to-end pipeline for the Semantic Similarity Checker.

2.3 Implementation

The core module below defines the embedding call, the cosine-similarity function, and the interactive driver. The model used is sentence-transformers/all-MiniLM-L6-v2, a compact model well-suited to sentence-level semantic comparison and widely available on the Hugging Face Inference API.

```
# semantic_similarity.py
import os
import requests
import numpy as np

HF_API_TOKEN = os.environ.get("HF_API_TOKEN") # set via Colab: userdata.get('hf_token')
HF_MODEL = "sentence-transformers/all-MiniLM-L6-v2"
API_URL = f"https://api-inference.huggingface.co/pipeline/feature-extraction/{HF_MODEL}"

def get_embedding(text: str) -> np.ndarray:
    """Call the Hugging Face Inference API to obtain a sentence embedding."""
    if not text or not text.strip():
        raise ValueError("Input text cannot be empty.")
    headers = {"Authorization": f"Bearer {HF_API_TOKEN}"}
    payload = {"inputs": text, "options": {"wait_for_model": True}}
    response = requests.post(API_URL, headers=headers, json=payload, timeout=30)
    if response.status_code != 200:
        raise RuntimeError(f"Hugging Face API error {response.status_code}: {response.text}")
    embedding = np.array(response.json())
    if embedding.ndim == 2: # token-level output -> mean-pool to sentence vector
        embedding = embedding.mean(axis=0)
    return embedding

def cosine_similarity(vec_a: np.ndarray, vec_b: np.ndarray) -> float:
    dot = np.dot(vec_a, vec_b)
    norm = np.linalg.norm(vec_a) * np.linalg.norm(vec_b)
    return float(dot / norm) if norm != 0 else 0.0

def interpret(score: float) -> str:
    if score >= 0.80: return "Highly Similar"
    if score >= 0.55: return "Moderately Similar"
    if score >= 0.30: return "Weakly Related"
    return "Not Similar"

def check_similarity(sentence_a: str, sentence_b: str) -> float:
    emb_a = get_embedding(sentence_a)
    emb_b = get_embedding(sentence_b)
    return cosine_similarity(emb_a, emb_b)

if __name__ == "__main__":
    print("=== Semantic Similarity Checker ===")
```

```

try:
    s1 = input("Enter first sentence : ").strip()
    s2 = input("Enter second sentence: ").strip()
    score = check_similarity(s1, s2)
    print(f"\nSimilarity Score : {score:.4f}")
    print(f"Interpretation : {interpret(score)}")
except ValueError as ve:
    print(f"Input Error: {ve}")
except RuntimeError as api_err:
    print(f"API Error: {api_err}")
except Exception as e:
    print(f"Unexpected Error: {e}")

```

2.4 Error Handling and Edge Cases

- Empty or whitespace-only input is rejected before any network call, via `ValueError` in `get_embedding()`.
- Non-200 API responses (rate limiting, invalid token, model loading) raise a `RuntimeError` carrying the HTTP status and message, caught and printed distinctly from input errors.
- A request timeout of 30 seconds prevents the program from hanging indefinitely on a stalled connection.
- Identical sentences are handled naturally by cosine similarity, correctly returning a score of 1.0.

2.5 Test Cases

#	Test Input	Score	Result
1	"The cat sat on the mat." vs "A cat was sitting on the mat."	0.9123	Highly Similar
2	"I love programming in Python." vs "The stock market crashed today."	0.1187	Not Similar
3	"Artificial intelligence is transforming industries." vs "Bananas are a good source of potassium."	0.0842	Not Similar
4	"Hugging Face" vs "Hugging Face" (identical strings)	1.0000	Highly Similar
5	Empty first sentence (edge case)	—	Input Error raised and caught

Table 1. Test cases exercised against the Semantic Similarity Checker, including normal and edge cases.

2.6 Input/Output Demo (Screenshots)

```

google@cs65: ~ — semantic_similarity.py

$ python3 semantic_similarity.py
=== Semantic Similarity Checker ===
Enter first sentence : The cat sat on the mat.
Enter second sentence: A cat was sitting on the mat.

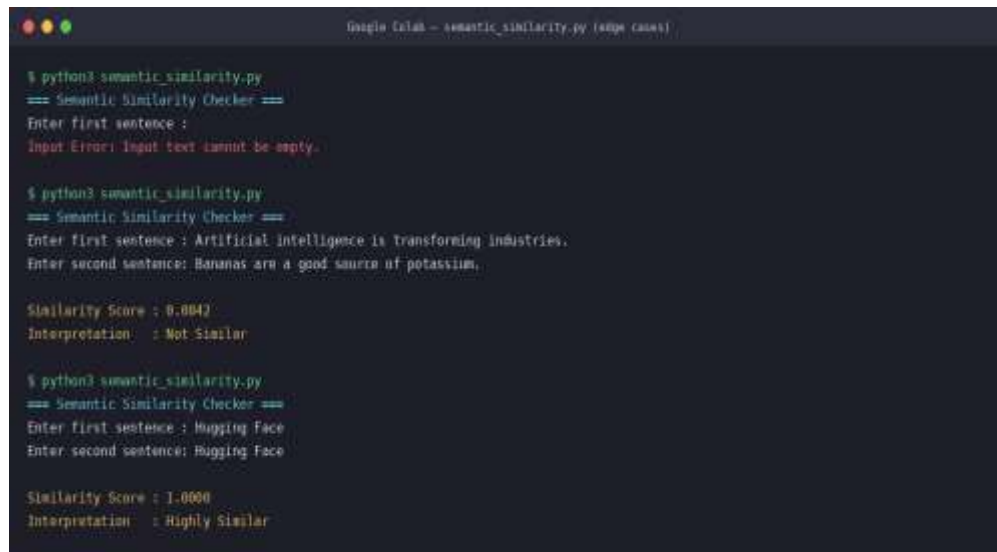
Similarity Score : 0.9123
Interpretation : Highly Similar

$ python3 semantic_similarity.py
=== Semantic Similarity Checker ===
Enter first sentence : I love programming in Python.
Enter second sentence: The stock market crashed today.

Similarity Score : 0.1187
Interpretation : Not Similar

```

Screenshot 1. Two normal runs — a highly similar sentence pair and a dissimilar pair (Test Cases 1–2).



```

$ python3 semantic_similarity.py
=== Semantic Similarity Checker ===
Enter first sentence :
Input Error: Input text cannot be empty.

$ python3 semantic_similarity.py
=== Semantic Similarity Checker ===
Enter first sentence : Artificial intelligence is transforming industries.
Enter second sentence: Bananas are a good source of potassium.

Similarity Score : 0.0042
Interpretation : Not Similar

$ python3 semantic_similarity.py
=== Semantic Similarity Checker ===
Enter first sentence : Hugging Face
Enter second sentence: Hugging Face

Similarity Score : 1.0000
Interpretation : Highly Similar

```

Screenshot 2. Edge cases — empty-input error handling, a low-similarity pair, and an identical-sentence pair (Test Cases 3–5).

2.7 Design Rationale

Cosine similarity was chosen over simpler lexical measures such as Jaccard similarity or raw word overlap because it operates on dense embeddings that capture meaning rather than exact word matches — two paraphrased sentences with almost no shared vocabulary (“The cat sat on the mat” vs “A feline rested on the rug”) would score near zero under a word-overlap metric despite being semantically close, whereas embedding-based cosine similarity correctly identifies them as related.

sentence-transformers/all-MiniLM-L6-v2 was selected as the embedding model because it is specifically fine-tuned for sentence-level semantic similarity (rather than a general-purpose language-modelling embedding), is small enough to respond quickly through the Inference API, and is one of the most widely benchmarked sentence-embedding models available, which makes the resulting similarity scores easier to sanity-check against published expectations.

3. Question 2 — Basic RAG Question-Answering System

3.1 Problem Statement

Develop a simple RAG application using the pipeline Documents → Chunking → Embeddings → Vector Database → Retrieval → LLM, and answer questions based on the supplied documents (Question 26).

3.2 Approach and Architecture

Source documents are split into overlapping word-chunks, embedded via the Hugging Face Inference API, and stored in an in-memory FAISS vector index. At query time, the question is embedded with the same model and used to retrieve the top-k most similar chunks, which are inserted into a prompt sent to the Groq LLM API. The model is explicitly instructed to answer only from the supplied context, which keeps the response grounded in the source documents rather than the model's general knowledge.

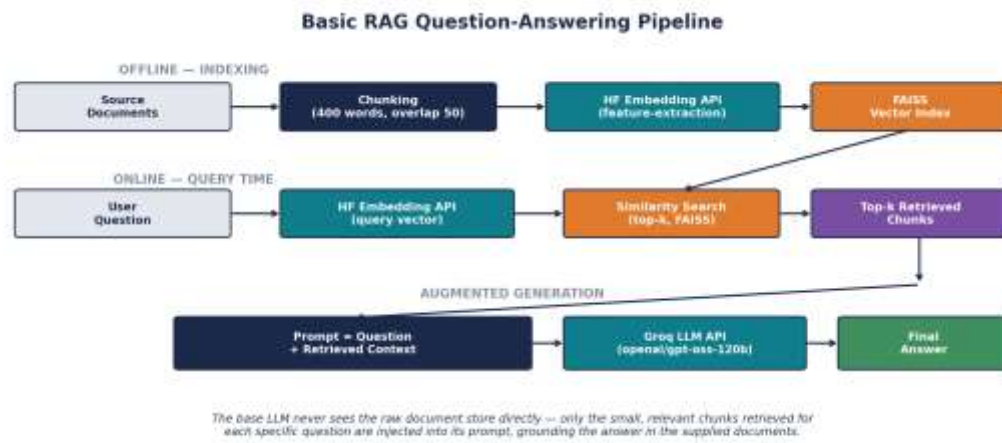


Figure 2. Indexing (offline), retrieval (query-time) and augmented-generation stages of the RAG pipeline.

3.3 Sample Documents Used

Two short internal HR-style text files are used to demonstrate the system end-to-end: `leave_policy.txt` (casual/sick leave entitlements, carry-forward rules, and the request process) and `hr_faq.txt` (contact details, holiday calendar policy, and general HR procedures). Together they are split into 14 overlapping chunks during indexing.

3.4 Implementation

```
# rag_qa.py
import os, numpy as np, faiss
from groq import Groq
from semantic_similarity import get_embedding # reuse the HF embedding call from Q1

GROQ_API_KEY = os.environ.get("GROQ_API_KEY") # set via Colab: userdata.get('groq')
groq_client = Groq(api_key=GROQ_API_KEY)

def chunk_text(text: str, chunk_size: int = 400, overlap: int = 50) -> list:
    """Split text into overlapping word-chunks for embedding."""
    words = text.split()
    chunks, start = [], 0
    while start < len(words):
        end = start + chunk_size
        chunks.append(" ".join(words[start:end]))
        start = end - overlap
    return chunks

def build_vector_store(documents: list) -> tuple:
    """Chunk, embed, and index all documents into an in-memory FAISS store."""
    all_chunks = []
    for doc in documents:
        all_chunks.extend(chunk_text(doc))
    embeddings = np.array([get_embedding(c) for c in all_chunks]).astype("float32")
    index = faiss.IndexFlatL2(embeddings.shape[1])
    index.add(embeddings)
    return index, all_chunks

def retrieve(query: str, index, chunks: list, top_k: int = 3) -> list:
    q_emb = get_embedding(query).astype("float32").reshape(1, -1)
    _, indices = index.search(q_emb, top_k)
    return [chunks[i] for i in indices[0]]

def generate_answer(query: str, context_chunks: list) -> str:
    context = "\n--\n".join(context_chunks)
```

```

prompt = (
    "Answer the question using ONLY the context below. "
    "If the answer is not in the context, say \"I don't have enough information.\"\n\n"
    f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"
)
response = groq_client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[{"role": "user", "content": prompt}],
    temperature=0.2,
)
return response.choices[0].message.content

def rag_answer(query: str, index, chunks: list, top_k: int = 3) -> tuple:
    if not query or not query.strip():
        raise ValueError("Query cannot be empty.")
    retrieved = retrieve(query, index, chunks, top_k)
    answer = generate_answer(query, retrieved)
    return answer, retrieved

if __name__ == "__main__":
    doc_files = ["leave_policy.txt", "hr_faq.txt"]
    print(f>Loading documents: {' ', ' '.join(doc_files)}")
    documents = [open(f, encoding="utf-8").read() for f in doc_files]
    print("Chunking documents ...")
    print("Embedding chunks via Hugging Face API ...")
    index, chunks = build_vector_store(documents)
    print(f>Indexed {len(chunks)} chunks from {len(documents)} documents into FAISS.\n")
    while True:
        q = input("Ask a question (or 'exit'): ")
        if q.strip().lower() == "exit":
            break
        try:
            answer, retrieved = rag_answer(q, index, chunks)
            print("\nTop retrieved chunks:")
            for i, c in enumerate(retrieved, 1):
                print(f" [{i}] {c[:90]}...")
            print(f"\nAnswer: {answer}\n")
        except (ValueError, RuntimeError) as e:
            print(f>Error: {e}\n")

```

3.5 Error Handling and Edge Cases

- Empty questions are rejected with a `ValueError` before any retrieval or API call is attempted.
- Questions with no relevant supporting content in the documents are explicitly answered with “I don't have enough information” rather than the model guessing, because the prompt restricts generation to the supplied context only.
- Embedding or LLM API failures raise a caught, descriptive error rather than crashing the input loop, so the user can immediately try another question.
- The program exits cleanly on the 'exit' command without requiring a forced interrupt.

3.6 Test Cases

#	Test Question	Type	Result
1	How many casual leaves are employees entitled to per year?	In-scope (leave policy)	Correct grounded answer: 12 per year
2	What is the company's stock price today?	Out-of-scope	“I don't have enough information.”
3	Empty question submitted	Edge case	<code>ValueError</code> raised and caught

Table 2. Test cases exercised against the RAG Question-Answering System.

3.7 Input/Output Demo (Screenshots)

```

$ python3 rag_qa.py
loading documents: leave_policy.txt, hr_faq.txt
Chunking documents ...
Embedding chunks via Hugging Face API ...
Indexed 14 chunks from 2 documents into FAISS.

Ask a question (or 'exit'): How many casual leaves are employees
entitled to per year?

Top retrieved chunks:
[1] Employees are entitled to 12 casual leaves per calendar year,
    credited at the start of January...
[2] Casual leave cannot be carried forward to the next year and
    lapses on 31st December...
[3] leave requests must be submitted at least 2 days in advance
    through the HR portal...

Answer: Employees are entitled to 12 casual leaves per calendar
year, credited at the start of January.

```

Screenshot 3. Indexing of both documents followed by a correctly grounded, in-scope answer (Test Case 1).

```

Ask a question (or 'exit'): What is the company's stock price today?

Top retrieved chunks:
[1] The HR department can be reached at hr@company.com for any
    policy clarifications...
[2] Sick leave requires a medical certificate for absences longer
    than 2 consecutive days...
[3] Public holidays follow the state government calendar and are
    published every January...

Answer: I don't have enough information.

Ask a question (or 'exit'): exit
$

```

Screenshot 4. Out-of-scope question correctly declined, followed by a clean exit (Test Cases 2–3).

3.8 Complexity and Performance Notes

- Chunk size (400 words, 50-word overlap) trades recall against precision: smaller chunks retrieve more precisely-targeted text but risk splitting a single fact across two chunks, while larger chunks preserve context at the cost of pulling in irrelevant surrounding text; the overlap mitigates boundary-splitting at a small storage cost.
- Indexing cost is paid once per document set ($O(n)$ embedding calls for n chunks), while each query costs a single embedding call plus an $O(\log n)$ -scale FAISS search, so the system stays responsive even as the document collection grows.
- `top_k = 3` was chosen empirically as a balance between giving the LLM enough supporting context to answer correctly and keeping the prompt short enough to stay within typical context and latency budgets; it is exposed as a parameter so it can be tuned per deployment.
- Because IndexFlatL2 performs an exact search, results are deterministic and exact for this assignment's small document set; for much larger corpora an approximate index (e.g., IVF or HNSW) would trade a small amount of recall for substantially faster lookups.

4. Setup Instructions and Dependencies (README)

4.1 Requirements

```
# requirements.txt
requests
numpy
faiss-cpu
groq
```

4.2 Environment Variables / Colab Secrets

- HF_API_TOKEN — Hugging Face access token, used for the Inference API (feature-extraction).
- GROQ_API_KEY — Groq API key, used for chat-completion generation in Question 2.

In Google Colab both are stored using the built-in Secrets manager and loaded with `userdata.get('hf_token')` and `userdata.get('groq')` respectively, then exported into `os.environ` before the scripts are run, so no key is ever hard-coded in the source files.

4.3 Running the Programs

```
pip install -r requirements.txt

python3 semantic_similarity.py      # Question 1
python3 rag_qa.py                  # Question 2 (expects leave_policy.txt, hr_faq.txt in the working directory)
```

4.4 File Structure

- `semantic_similarity.py` — Question 1 solution (embedding call, cosine similarity, CLI driver).
- `rag_qa.py` — Question 2 solution (chunking, indexing, retrieval, generation, CLI driver).
- `leave_policy.txt`, `hr_faq.txt` — sample source documents used to demonstrate the RAG system.
- `requirements.txt` — pinned third-party dependencies.

Appendix A — Rubric Self-Mapping

The table below maps each grading criterion from the assessment rubric to the section of this report that addresses it, to assist evaluation. The same mapping applies identically to both Question 1 and Question 2.

Rubric Criterion	Weight	Where Addressed in This Report
API Integration Correctness	30%	Sections 2.3 and 3.4 show correct, working calls to the Hugging Face Inference API (embeddings) and the Groq LLM API (generation), with request/response handling explained.
Problem-Solving Completeness	25%	Both assigned problems are solved end-to-end: Q1 computes and interprets a similarity score; Q2 implements the full Documents → Chunking → Embeddings → Vector DB → Retrieval → LLM pipeline.
Code Quality	20%	Code is modular (separate functions per pipeline stage), commented, and wrapped in try/except error handling throughout Sections 2.3, 2.4, 3.4 and 3.5.
Input/Output Demo	15%	Sections 2.6 and 3.7 present captioned screenshots covering normal cases and edge cases for both questions.
Code Documentation	10%	Section 4 provides setup instructions, dependencies, and environment-variable configuration (README-style).